

Contents

CerviQ User Documentation	1
1. Overview	1
2. Requirements and build	1
3. Main menu and setup screens	1
4. Watch Agents	2
5. Play vs Agents	4
6. Scenarios, capacities, and food	6
7. Selectable agents	6
8. Large-map viewport and indicators	7
9. Typical workflows	8
10. Advanced operation: training and evaluation	8
11. Troubleshooting and user-visible limitations	9

CerviQ User Documentation

1. Overview

CerviQ is a real-time grid game for multiple continuously moving worms. On each game tick, every living worm turns left, continues straight, or turns right, then all worms move simultaneously. Food grows surviving worms. Collisions with walls, poison, bodies, or other heads can kill them.

The desktop application provides two modes:

- **Watch Agents:** configure 2-9 AI-controlled worms and inspect the running simulation, history, and Q-learning diagnostics.
- **Play vs Agents:** control Worm 1 against one or more independently selected AI opponents.

The actual roster limit depends on how many valid predefined starts the chosen scenario provides.

2. Requirements and build

You need:

- Stack;
- a graphical desktop with OpenGL support for the Gloss window;
- the repository's model files under `data/models/`.

`stack.yaml` pins Stackage snapshot `lts/24.48.yaml`. The first build may download the required compiler and packages.

Run all commands from the repository root. Build all configured executables with:

```
stack build
```

Run the automated unit-test suite with:

```
stack test
```

Start the graphical application with:

```
stack run cerviQ
```

The working directory matters because the application loads seven Q-table files through relative paths. The Gloss window is initially 1400 x 760 pixels. New Play and Watch sessions start paused at 3 ticks per second.

3. Main menu and setup screens

The main menu contains **Watch Agents**, **Play vs Agents**, and **Quit**.



Figure 1: CerviQ main menu. *Watch Agents* opens AI simulation and diagnostics, while *Play vs Agents* starts a human-controlled game.

Key	Main menu action
Up / Down	Move the selection; selection wraps
Enter	Open the selected setup screen or quit

Both setup screens use dynamic rows.

Key	Setup action
Up / Down	Move through map, controller, add/remove, Start, and Back rows
Left / Right	Cycle the selected map or controller; values wrap
Enter	Activate Add, Remove, Start, or Back when that row is selected
Esc	Return to the main menu

Left and Right have no effect on action rows. Disabled Add or Remove operations leave the setup unchanged. Changing to a lower-capacity scenario trims either stored roster to that scenario's capacity; switching back does not restore the removed rows automatically.

4. Watch Agents

Watch setup starts with two controller rows. Select **+** **Add agent** to append a worm, or **-** **Remove last** to remove the final row. At least two worms must remain. Each row cycles independently through every registered controller.

The maximum Watch roster is:

`min(9, selected scenario's predefined worm starts)`

Added rows initially use **Safe Greedy Food**. The default two-agent setup is **Q-learning V4 Reformed 30k** **+** **fallback** and **Safe Greedy Food**. Starting a session assigns the selected controllers to the predefined worms in order and gives every worm a stable, distinct color.

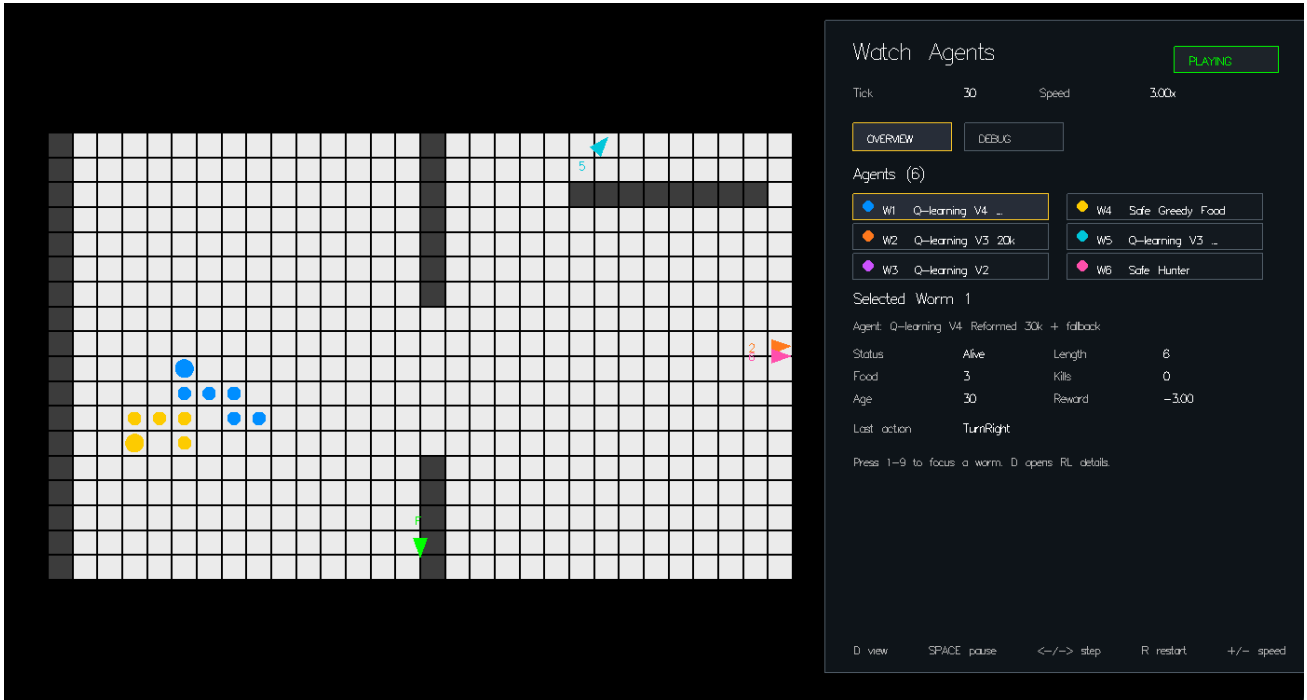


Figure 2: Multi-worm Watch Agents session. Each worm has its own controller and color, while the HUD summarizes the configured roster and current statistics.

Watch controls

Key	Action
Space	Pause or resume automatic simulation
Right arrow	Pause and advance one tick, or replay the next stored future snapshot
Left arrow	Pause and move to the previous stored snapshot
D or d	Toggle Overview and Debug HUD pages
1-9	Select a configured worm with that numeric ID
+ or =	Multiply speed by 1.5, up to 20 ticks/s
-	Divide speed by 1.5, down to 0.25 ticks/s
R or r	Restart the current session paused and clear both history directions
Esc	Return to Watch setup with selections retained

Number keys that do not identify a configured worm have no effect. Watch stops automatically only when no worm remains alive. A sole survivor therefore keeps moving until it also dies, or until the user pauses, restarts, or leaves.

History and restart

History is made of complete `GuiSnapshot` values, not inverse game operations. After stepping left, Right replays the stored state before computing any new controller decisions. Restart restores the session's stored initial state, including its original randomized food positions, and does not draw a new food layout. Return to setup and start again to create a new randomized session.

Overview and Debug

Overview shows all configured agents, with a two-column roster for five or more worms. The selected worm section shows its agent, alive status, length, food, kills, age, last action, and last transition reward.

Debug is focused only on the selected worm. Heuristic controllers have no Q-learning provider, so they show **No Q-learning debug data**. Learned agents show their version-specific encoded state, all three raw Q-values, and actions tied for the raw maximum. V2-V4 add internal diagnostic values. V4 includes action quality, path, robust-space, tail-connectivity, repetition, and known-Q information.

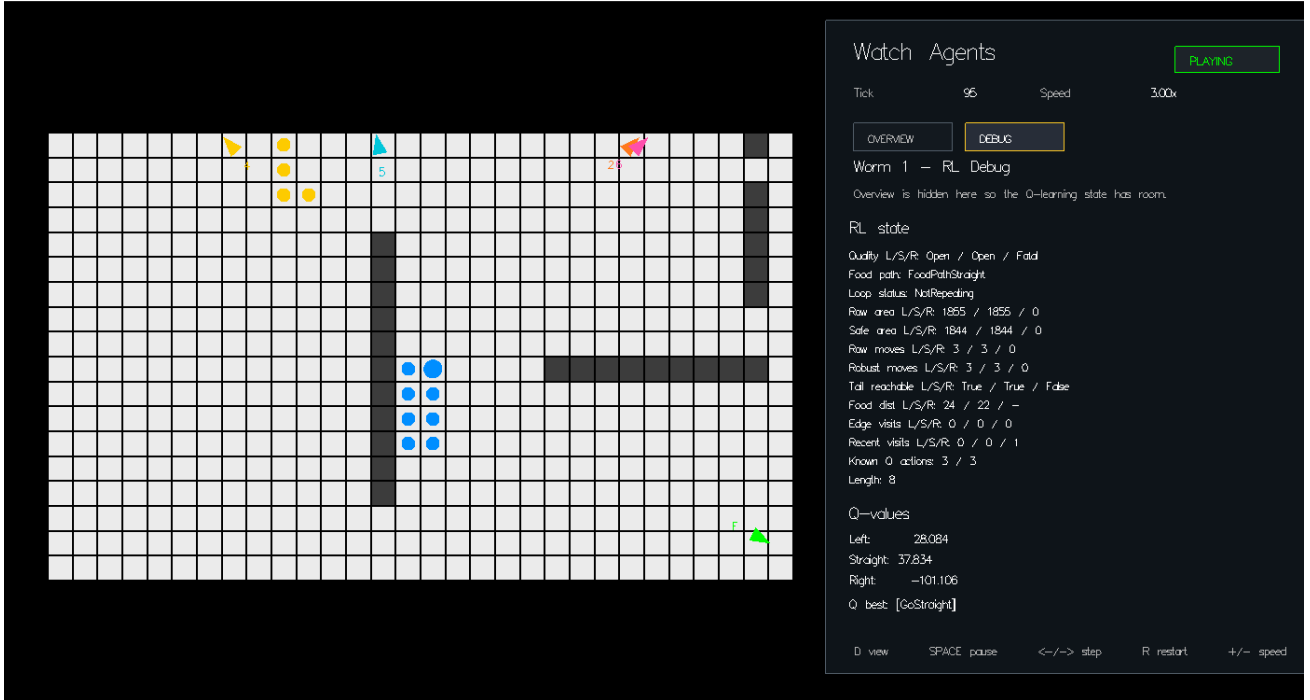


Figure 3: Debug HUD for Q-learning V4 Reformed 30k + fallback. The panel exposes the encoded RL state, raw Q-values, food-path information, action-quality diagnostics, topology information, repetition state, and known-Q status used to inspect agent decisions.

For a + **fallback** controller, **Q best** is not necessarily the action that is executed. The display reports unfiltered table maxima, while V3 and V4 fallback policies can remove unsafe candidates before learned values are compared.

5. Play vs Agents

Play setup always reserves the first predefined worm, shown as Worm 1, for the human. It starts with one AI opponent. Use + **Add opponent** and - **Remove last** to configure the roster; each opponent controller can be selected independently.

At least one AI opponent is required. The maximum is:

```
min(8, selected scenario's predefined worm starts - 1)
```

Added opponents initially use **Safe Greedy Food**. Worm 1 is blue and every AI opponent receives a different roster color. All AI controllers are queried on every game tick, and their actions are submitted together with the human action to the simultaneous game engine.

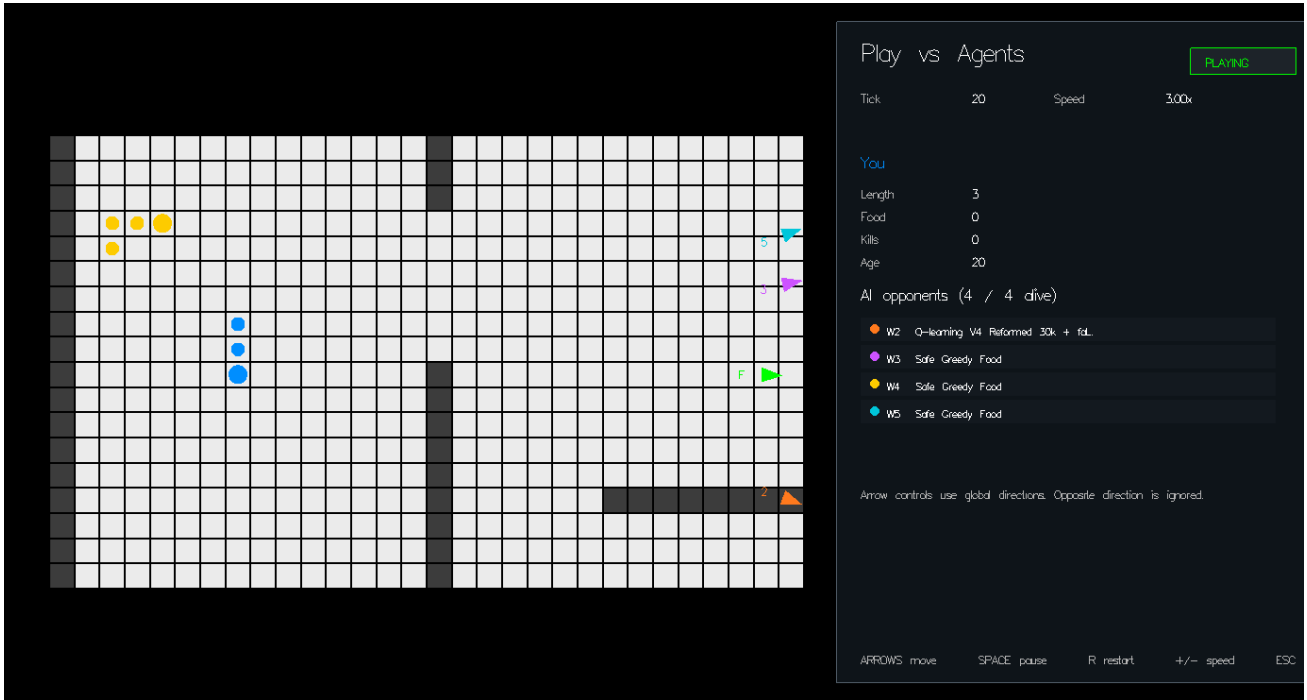


Figure 4: *Play vs Agents* with multiple AI opponents. Worm 1 is controlled by the player, while every opponent is assigned an independent controller and roster color.

Play controls

Key	Action
Up	Request absolute direction North
Right	Request absolute direction East
Down	Request absolute direction South
Left	Request absolute direction West
Space	Pause or resume while the game result is still running
+ or =	Multiply speed by 1.5, up to 20 ticks/s
-	Divide speed by 1.5, down to 0.25 ticks/s
R or r	Restart the current session paused
Esc	Return to Play setup with selections retained

The arrow keys request absolute directions on the map. The game converts the request into the worm-relative action used by the engine. A direct 180-degree reversal is illegal and ignored. If several valid directions are pressed before the next tick, only the last pending action is used. The pending action resets to **GoStraight** after the tick.

Play has no manual single-tick or rewind keys; Left and Right are movement controls. Restart restores the same randomized starting food layout. Starting a new session from setup produces another layout.

Results and HUD

Play ends according to all configured opponents, not only the first one.

Human state	Opponent state	Result
Alive	At least one alive	Game continues
Alive	All dead	PlayerWon / YOU WIN

Human state	Opponent state	Result
Dead	At least one alive	PlayerLost / GAME OVER
Dead	All dead	PlayDraw / DRAW

The HUD shows human length, food, kills, and age. Its opponent roster shows each worm ID, controller name, color, and alive/dead appearance, using two columns when more than four opponents are configured. A kill is credited only when a worm dies by hitting another worm's non-head body. Head-to-head deaths do not award kills.

6. Scenarios, capacities, and food

Scenario	Size	Description	Watch worms	Play AI opponents
Arena	20 x 10	Open baseline used during Q-learning development	2-6	1-5
Cave	30 x 15	Chambers, obstacles, and alternative routes	2-8	1-7
Corridors	30 x 15	Chokepoints and restricted passages	2-6	1-5
Large Cave	60 x 36	Large multi-chamber viewport map	2-9	1-8

All map layouts and starts are deterministic. Large Cave is constructed from fixed coordinate rules, not generated procedurally.

Adaptive interactive food

When Play or Watch starts, authored food is cleared. The application computes a target from the scenario and the configured starting worm count:

`min(scenario maximum, scenario base + (worm count - 1) div 2)`

Scenario	Base	Maximum	Target by total starting worms
Arena	2	4	2:2, 3:3, 4:3, 5:4, 6:4
Cave	2	5	2:2, 3:3, 4:3, 5:4, 6:4, 7:5, 8:5
Corridors	2	4	2:2, 3:3, 4:3, 5:4, 6:4
Large Cave	3	7	2:3, 3:4, 4:4, 5:5, 6:5, 7:6, 8:6, 9:7

The target is fixed for the lifetime of the session, even after worms die. Consumed food is replenished back to that target.

Food is selected uniformly at random from legal empty cells. Interactive spawning first prefers cells whose Manhattan distance from every living worm head is at least 3. If no such cell exists, it falls back to any legal free cell. This is only a near-head safeguard; it is not a fairness optimizer, BFS placement, or farthest-point strategy.

7. Selectable agents

`AgentRegistry.loadAgentOptions` exposes these exact menu names:

Group	Agents
Heuristic	Random; Safe Random; Greedy Food; Safe Greedy Food; Safe Hunter
V1-V2	Q-learning V1; Q-learning V2
V3	Q-learning V3 10k; Q-learning V3 20k; Q-learning V3 20k + fallback; Q-learning V3 Diverse 30k; Q-learning V3 D
V4	Q-learning V4 Diverse 30k; Q-learning V4 Diverse 30k + fallback; Q-learning V4 Reformed 30k; Q-learning V4 Re

The heuristic agents behave as follows:

- **Random:** chooses any relative action uniformly.
- **Safe Random:** chooses among locally safe actions when possible.
- **Greedy Food:** reduces Manhattan distance to the nearest food without a safety check.
- **Safe Greedy Food:** combines immediate safety with greedy food distance.
- **Safe Hunter:** safely approaches the nearest living enemy head.

Safety checks consider all other living bodies, but they do not predict the actions those opponents will choose. V4 performs a richer one-step threat approximation over all living opponents; it is still not adversarial minimax.

The supplied training path uses each scenario's two default worms. The current V4 runtime can analyze multiple opponents, but the included model was not separately retrained for large 6-9 worm matches. Such matches therefore contain more unseen or aliased situations than the original one-versus-one training environment.

8. Large-map viewport and indicators

A map uses viewport rendering only when it is wider than 30 cells or taller than 18 cells. Among the built-in scenarios, this means Large Cave. Other maps are rendered in full.

The viewport is at most 30 x 18 cells:

- Play follows the human worm.
- Watch follows the worm selected with 1-9.
- Near an edge, the viewport is clamped to the map boundary.
- Selecting a dead Watch worm leaves focus at that worm's final head position.

Cropping affects rendering only. Controllers and Q-learning encoders continue to receive the complete `GameState` and map.

Off-screen living worms appear as colored edge arrows labeled by worm ID. Play shows every living AI opponent; Watch shows every living worm except the selected one. A green **F** arrow points toward the geometrically nearest food when it is outside the viewport. This display choice is separate from V4's path-aware food analysis.

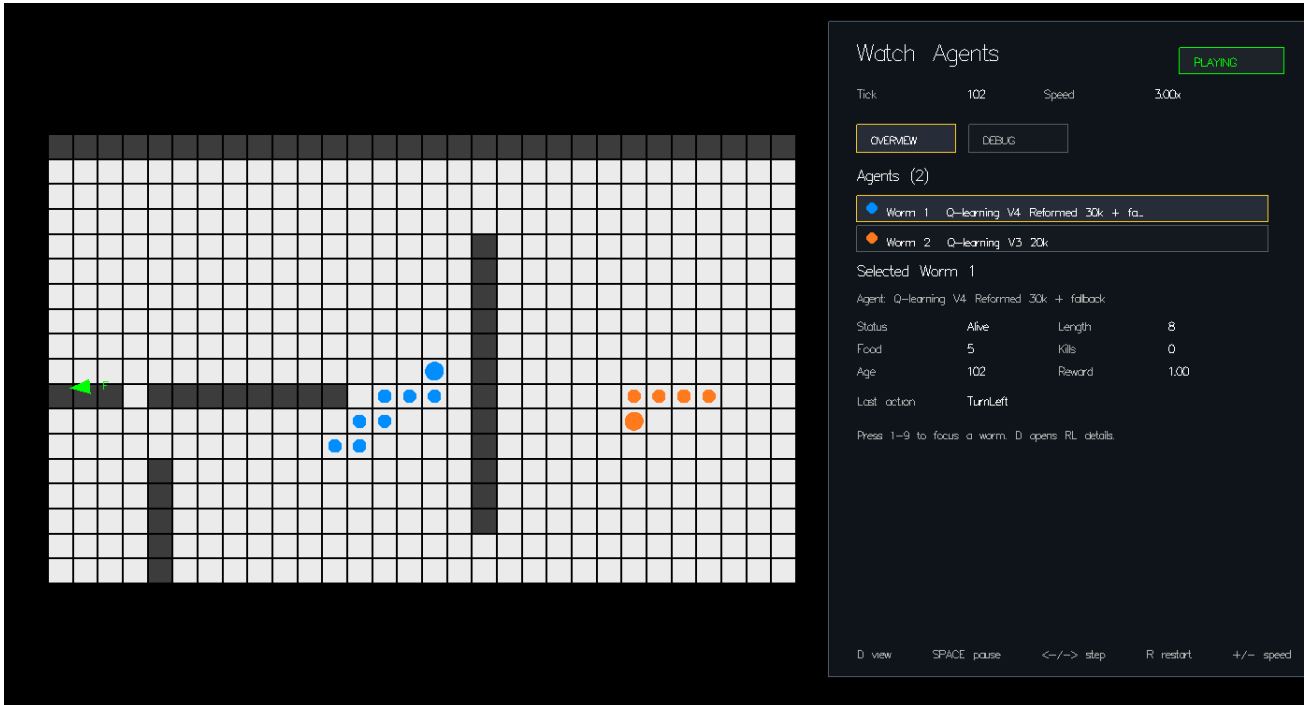


Figure 5: Large Cave viewport. The camera follows the focused worm, while colored edge indicators show off-screen living worms and the green F marker indicates food outside the visible region.

9. Typical workflows

Configure a multi-agent Watch session

1. Run `stack run cerviq` and choose **Watch Agents**.
2. Choose Cave or Large Cave.
3. Add agents and select each controller independently.
4. Start, press Space, and use 1-9 to change focus.
5. Press D on a Q-learning worm to inspect its encoded state and Q-values.

Play against different opponents

1. Choose **Play vs Agents**.
2. Keep Worm 2 as **Q-learning V4 Reformed 30k + fallback**.
3. Add opponents such as **Safe Hunter** and **Safe Greedy Food**.
4. Start, press Space, and steer Worm 1 with the arrow keys.

Inspect decisions one tick at a time

In Watch, remain paused and press Right once per tick. Use Left to revisit older snapshots, switch the selected worm with a number key, and compare **Last action**, diagnostic state, and raw Q-values.

10. Advanced operation: training and evaluation

Reproduce the final V4 training recipe

Run the dedicated executable from the repository root:

```
stack run cerviq-train-v4
```

It has no command-line arguments. The fixed source configuration performs 30,000 episodes, prints progress every 200 episodes, and writes:

```
data/models/qlearning_v4_reformed_diverse_30k.txt
data/models/qlearning_v4_reformed_diverse_30k_stats.txt
```

Training starts from an empty Q-table and writes only after the complete run; it does not checkpoint or resume. The sampler uses the original two-worm versions of Arena, Cave, and Corridors, swaps their two starting sides with 50% probability, clears authored food to one random item, and does not use Large Cave or the optional multi-worm starts.

Run a configurable experiment

The shorter configurable trainer supports V1-V4 and exposes its current options through:

```
stack run cerviq-train -- --help
```

For example, this trains V4 for 500 episodes and writes a model plus raw, CSV, and summary statistics alongside it:

```
stack run cerviq-train -- v4 --episodes 500 --output data/models/demo_v4.txt
```

Pairwise evaluation compares two controllers on an original two-worm scenario:

```
stack run cerviq-evaluate -- \
  --agent "Q-learning V4 Reformed 30k + fallback" \
  --opponent "Safe Greedy Food" \
  --scenario Arena \
  --episodes 100
```

Training and evaluation use unseeded randomness, so repeating a command does not reproduce the exact episode sequence. See Training and evaluation for all options, output formats, direct model references, and metric semantics.

11. Troubleshooting and user-visible limitations

- **The game opens paused.** Press Space. In Watch, Right also advances one paused tick.
- **A direct reverse arrow does nothing.** Worms cannot turn 180 degrees.
- **The Add row does nothing.** The current scenario capacity has been reached.
- **A numbered Watch key does nothing.** That worm ID is not configured.
- **Startup reports a missing or invalid Q-table.** Run from the repository root and keep the seven files loaded by `AgentRegistry.loadAgentOptions` in `data/models/`.
- **No window appears.** Gloss requires a working graphical desktop and OpenGL; a headless session is insufficient.
- **Results differ between starts.** Agent tie-breaking and interactive food use process-global randomness with no exposed seed option.
- **Poison never appears.** The engine supports and renders it, but no built-in scenario contains a poison tile.
- **There are no graphical-application CLI options for map, roster, speed, or controllers.** Configure them in the GUI. The dedicated final V4 recipe is fixed, while `cerviq-train` and `cerviq-evaluate` expose experiment options.